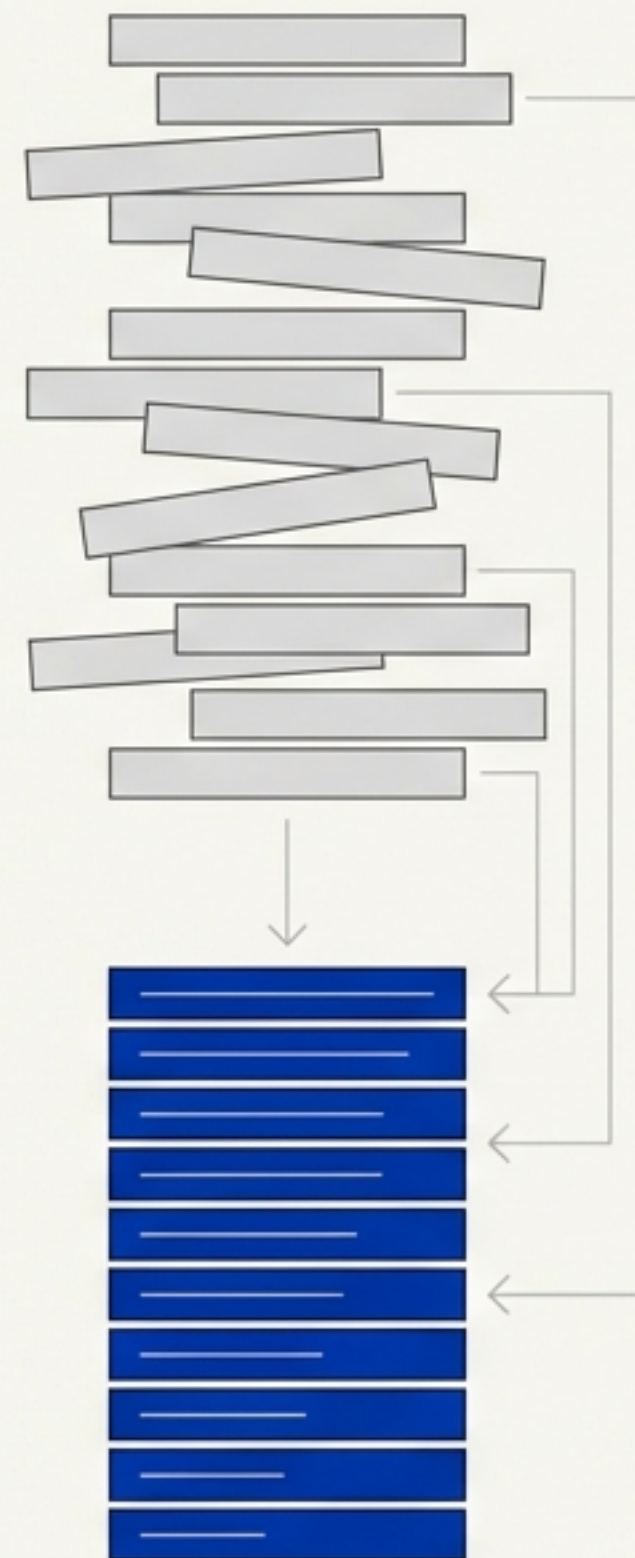


Mastering Array Algorithms

Optimization Patterns: In-Place Modification & Sliding Window

LeetCode 448 & 487



Based on the Arrays Playlist by CTO Bhaiya

The Optimization Roadmap

Module 1: Space Optimization

Problem: Find All Numbers Disappeared in an Array (LeetCode 448)

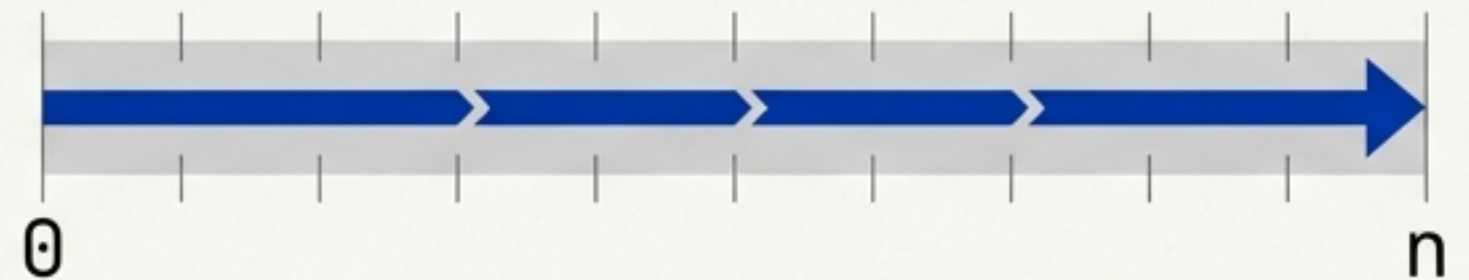
Target: $O(1)$ Space



Module 2: Time Optimization

Problem: Max Consecutive Ones II (LeetCode 487)

Target: $O(n)$ Time



Goal: Moving from Brute Force Intuition to Optimal Execution.

Case Study A: The Disappearing Numbers

LeetCode 448

- **Input:** Array of size n . Integers range $[1, n]$.
- **Twist:** Some elements appear twice, others appear once.
- **Objective:** Find all numbers in $[1, n]$ missing from the array.
- **Constraint:** $O(n)$ Time, $O(1)$ Space.

Input

4	3	2	7	8	2	3	1
0	1	2	3	4	5	6	7



Output

5	6
---	---

Indices 5 and 6 were never mapped.

The Intuition: Using a HashSet

Input Array

4
3
2
7
8
2
3
1

HashSet (External Memory)

• {1 • 3
 • 2
• 4 • 7 • 8}

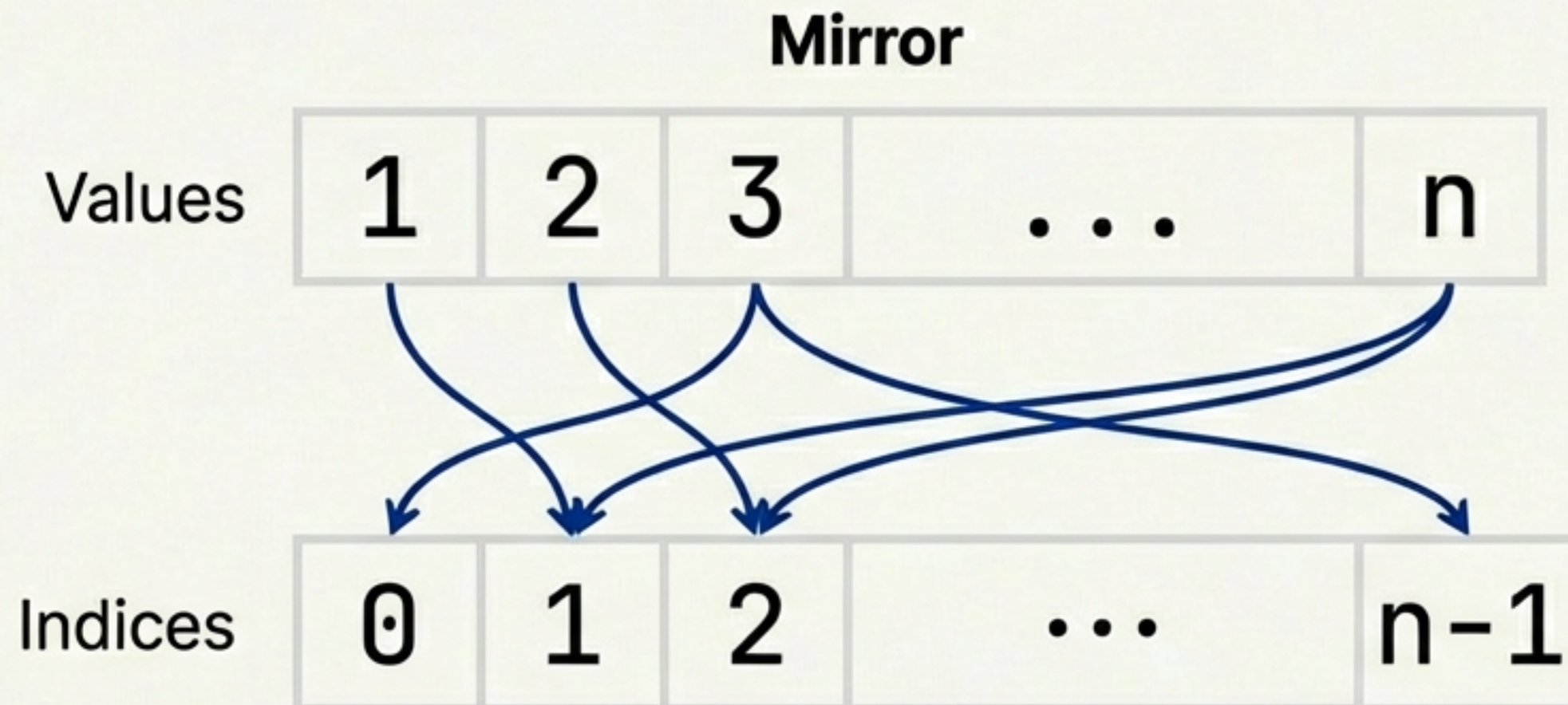
1. Add all elements to a HashSet. 2. Iterate 1 to n. Check if present in Set. 3. Missing items = Result.

Verdict: Inefficient. Space Complexity is $O(n)$. We need to eliminate the external **Set**.

The Insight: The Array is its Own Checklist

Concept: Values map to Indices.

Since values are in range $[1, n]$ and indices are $[0, n-1]$, we can use the array to track its own state.



Negative Marking Strategy

If we see value x , go to index $x-1$ and flip that number to negative.

Negative = Visited.



Visualizing the Negative Mark

Step 1
Inter Regular

Index 0 (Value 4)



4	3	2	7	8	2	3	1
---	---	---	---	---	---	---	---



Step 2
Inter Regular

4	3	2	7	8	2	3	1
---	---	---	---	---	---	---	---



Mark index $(4-1) = 3$ as negative.

Step 3
Inter Regular

4	3	2	-7	8	2	3	1
---	---	---	----	---	---	---	---

IMPORTANT

Always take `Math.abs(nums[i])`. A number might have been negated by a previous step, but we still need its original value to find the index.

Retrieving the Missing Numbers

-4	-3	-2	-7	8	2	-3	-1
0	1	2	3	4	5	6	7

Index 4 is positive ->
Value 5 is missing.

Index 5 is positive ->
Value 6 is missing.

Result List:

[5, 6]

Logic:

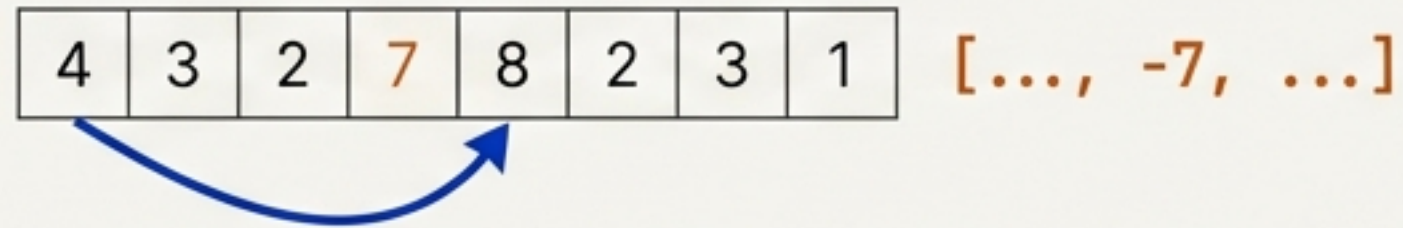
If $\text{nums}[i] > 0$, then $i + 1$
was never seen in the
array.

Summary: In-Place Modification

The Algorithm

1. Iterate & Mark:

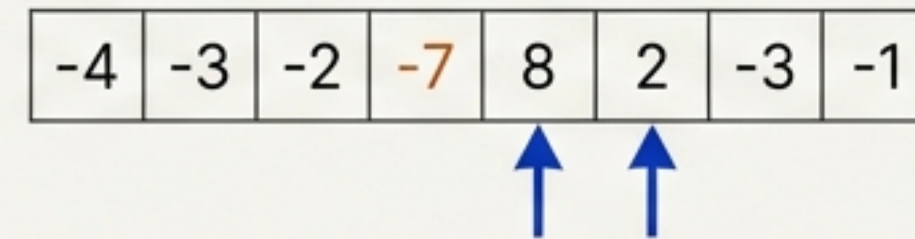
```
int idx = Math.abs(nums[i]) - 1;  
if (nums[idx] > 0) nums[idx] = -nums[idx];
```



Mark index ($\text{Math.abs}(\text{nums}[i]) - 1$) as negative.

2. Iterate & Collect:

```
if (nums[i] > 0) result.add(i + 1);
```

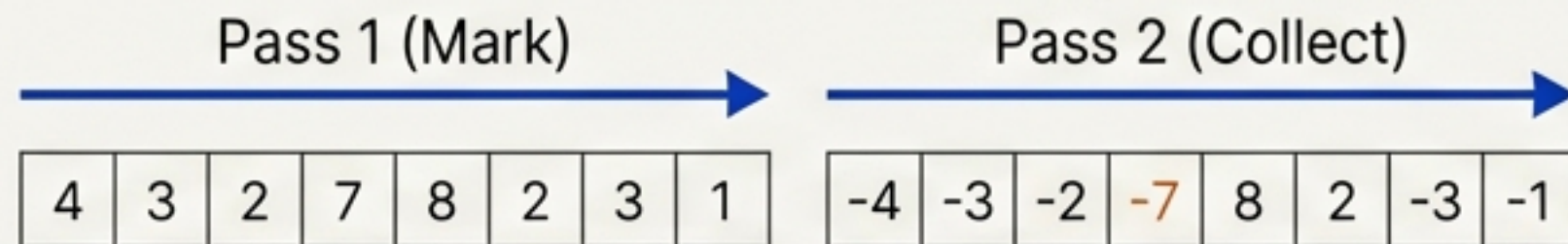


Result: [5, 6]

Positive values indicate missing numbers (index + 1).

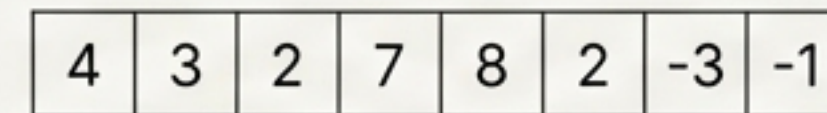
Complexity

Time Complexity: $O(n)$

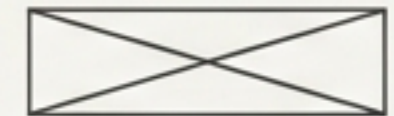


Visual: 2 Linear Passes.

Space Complexity: $O(1)$



Input Array
(Used for State)



External Structures
(e.g., Hash Set, Extra Array)

Visual: No external structures.

Key Takeaway: Use the **input array** to store boolean state (**Positive/Negative**).

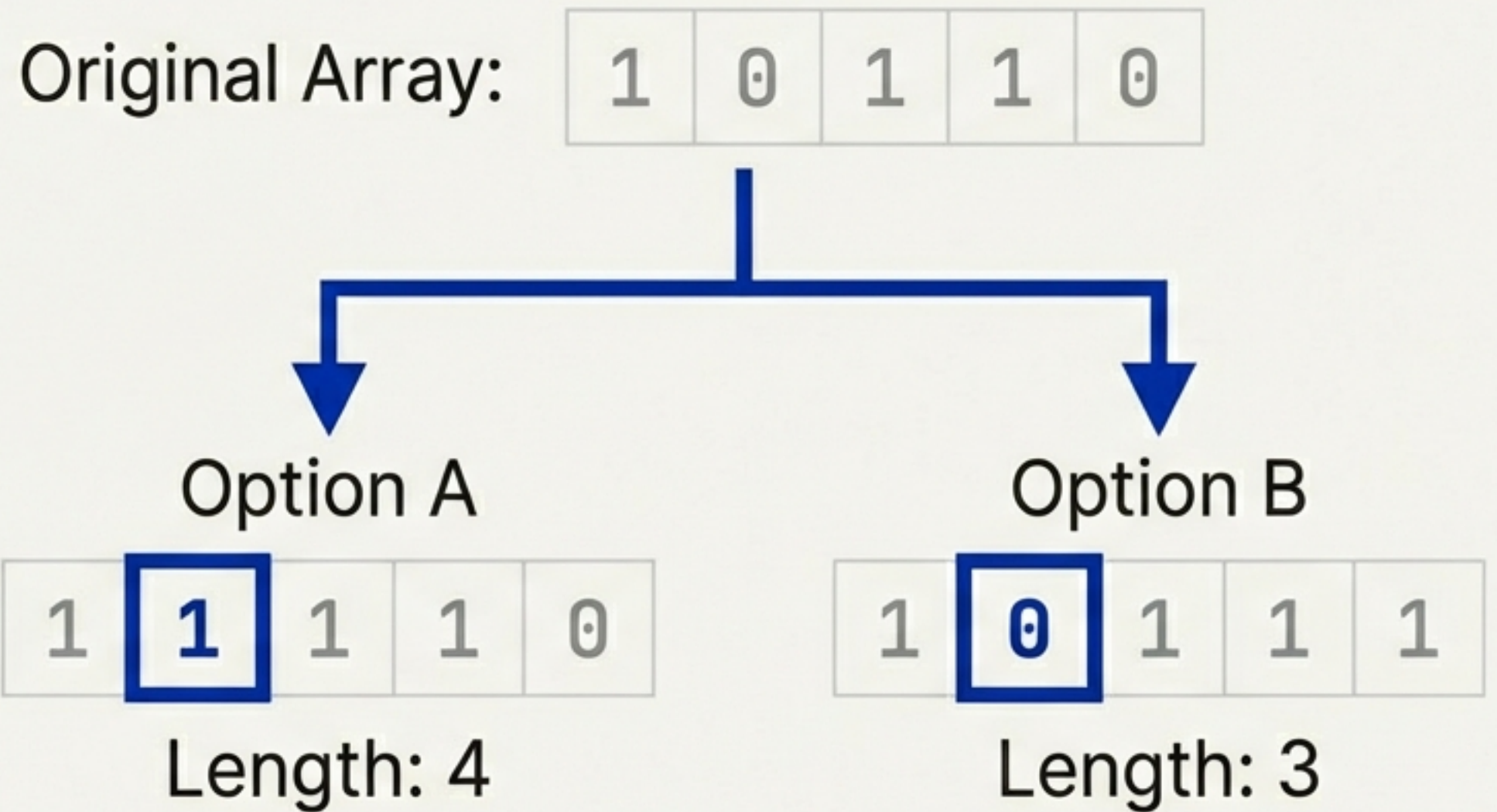
Case Study B: The Flexible Window

LeetCode 487: Max Consecutive Ones II

Input: Binary Array [0, 1].
Size up to 10^5 .

Challenge: Find max consecutive 1s.

The Twist: You can flip at most one '0' to '1'.

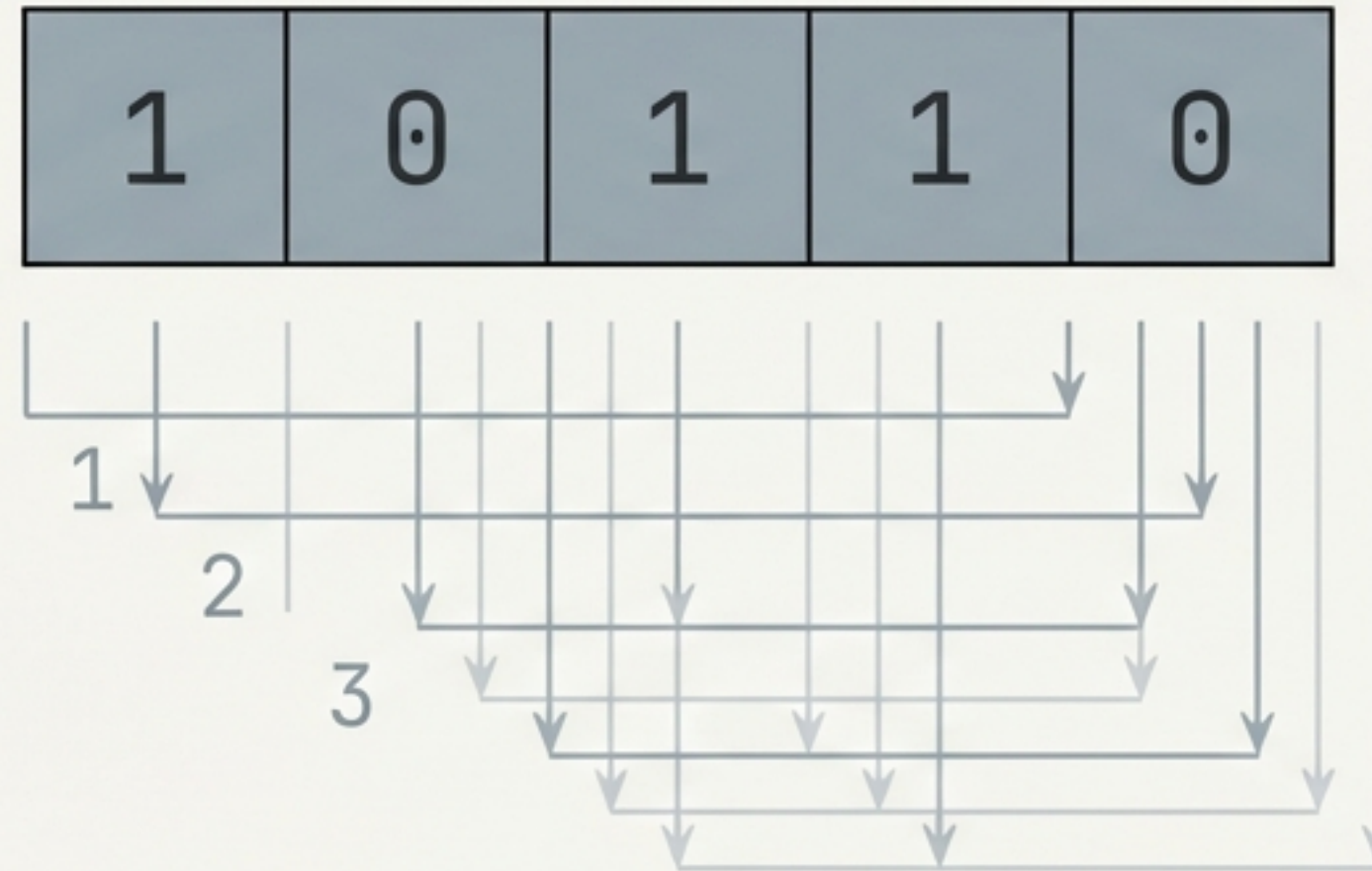


Max Answer: **4**

The Brute Force Approach

Method:

- Check every possible sub-array range using nested loops.
- Inner Loop (j) expands until it hits a second zero.



Inefficiency: We are re-calculating overlapping sub-arrays.
Complexity: $O(n^2)$. For $N=10^5$, this will Time Out (TLE).

The Insight: Sliding Window



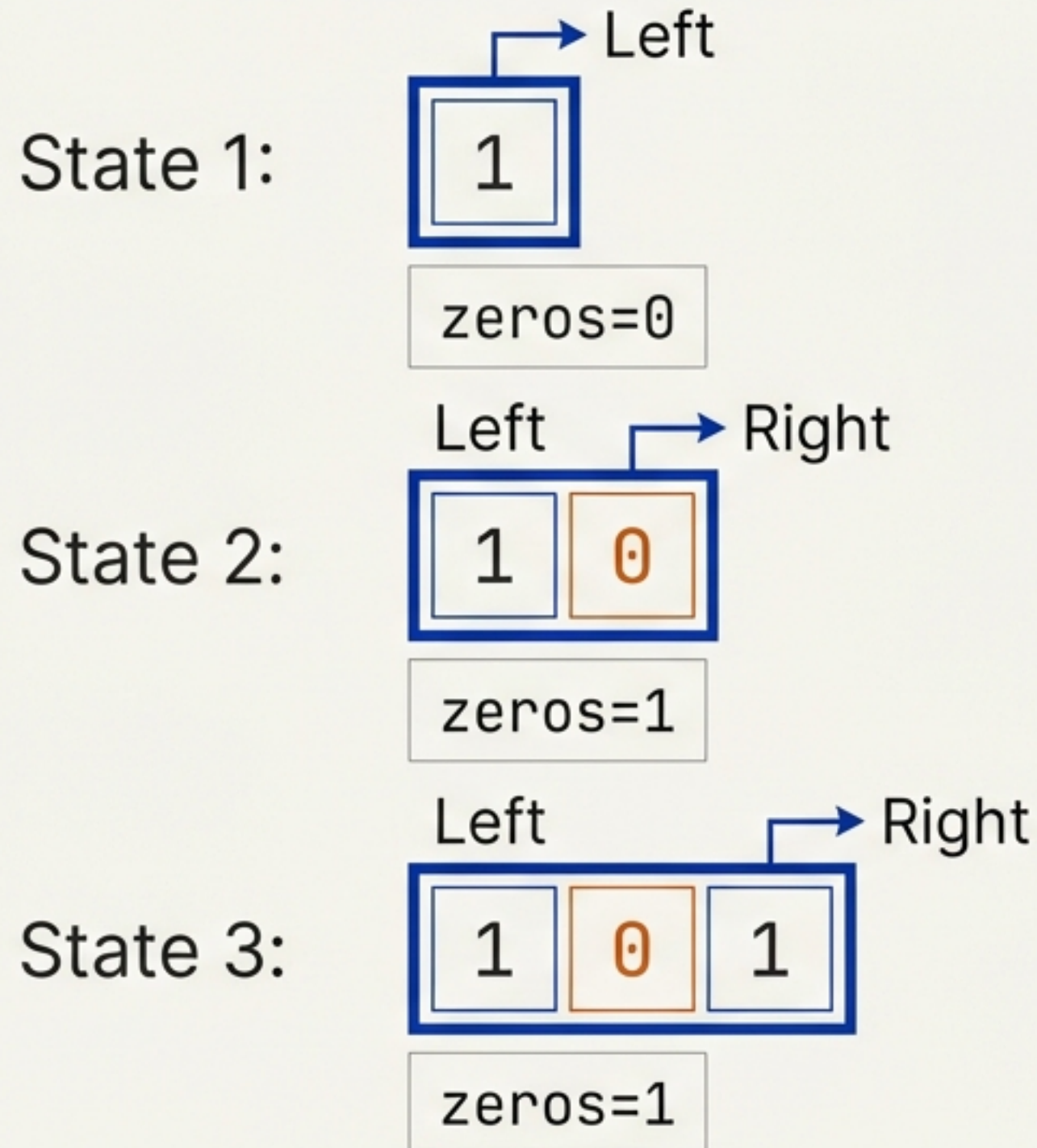
Concept: Maintain a dynamic window `[left, right]` that represents the current valid sequence.

Rule: The window is **VALID** if it contains **at most one "0"**.

————— **Expand** (Right Pointer) to find more ones. —————→

←————— Contract (Left Pointer) ONLY when **invalid** (**zero count > 1**). —————

Mechanics: Expansion (The Right Pointer)



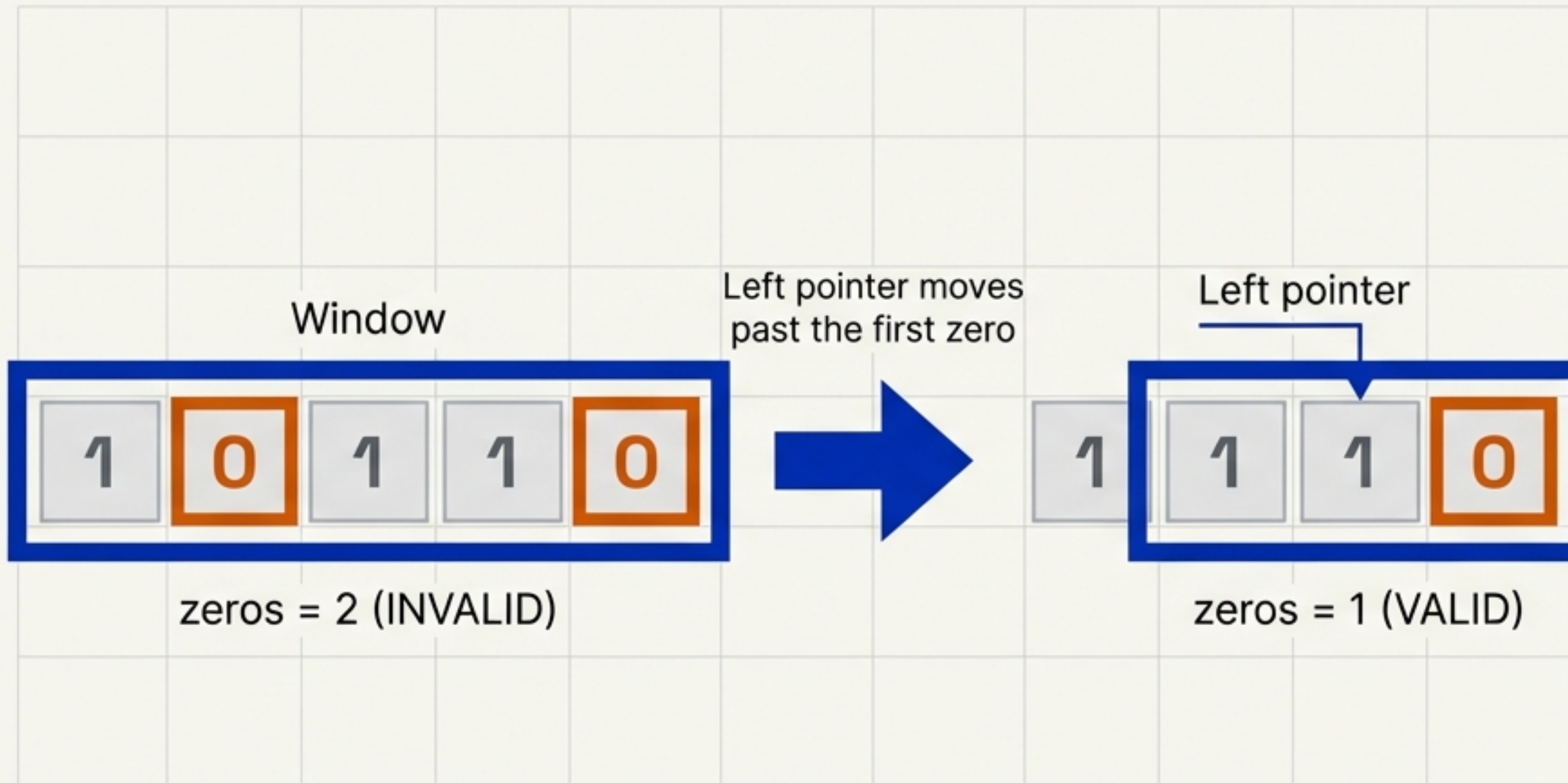
Action:

Move Right pointer forward every step.

Logic:

- If `nums[right]` is 0:
`zeros++`
- Update Max Length:
`max(current_max, right - left + 1)`

Mechanics: Contraction (The Left Pointer)



Trigger: When zeros > 1 , the window is broken.

Fix: Increment Left pointer to shrink from the back.

Update: If the number leaving the window was a '0', decrement zero count.

Narrative: Don't restart. Just trim the tail.

Summary: The Dynamic Window" in Space Grotesk

Bold, tight tracking, Deep Charcoal (#222222)

Pseudocode Logic

Inter Regular, Slate Grey (#444444)

```
while (right < n) {  
  if (nums[right] == 0) zeros++;  
  while (zeros > 1) {  
    if (nums[left] == 0) zeros--;  
    left++;  
  }  
  max = Math.max(max, right - left + 1);  
  right++;  
}
```

Performance Metrics

Inter Regular, Slate Grey

Time: **$O(n)$**

(Each element is visited at most twice: once by Right, once by Left).

Inter Regular, Slate Grey

Space: **$O(1)$**

(Only pointers and counters).

Inter Regular, Slate Grey

Verdict: Solves 10^5 elements instantly.

Patterns to Master

Data as State

From Problem 448

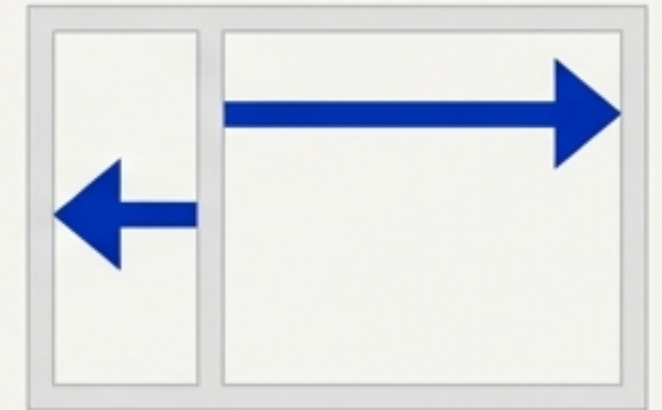
Don't fear modifying the input. Use indices to map presence and save space.



Dynamic Windows

From Problem 487

Avoid nested loops. Shrink from the left to maintain validity without resetting.



Call to Action: Join the Discord for full source code.
Keep Coding. See you in the next one. — CTO Bhaiya